



SmartPot

DOCUMENTACIÓN DEL COMPONENTE

SmartPot-DB

La memoria de SmartPot

MongoDB 8 con validación \$jsonSchema en diez colecciones, usuario de la aplicación con permisos mínimos, datos demo opcionales y migración idempotente para bases existentes.



DOCUMENTO
SmartPot-DB

VERSIÓN
1.1 · octubre 2026

EQUIPO
SmartPotTech

PROYECTO
smartpot.app



DOCUMENTO

Ficha del documento

CAMPO	VALOR
Proyecto	SmartPot · smartpot.app
Componente	SmartPot-DB
Versión	1.1 · octubre 2026
Alcance	Colecciones y validadores, inicialización, migración, datos demo, configuración, pruebas y operación
Documentación de la plataforma	Documentación técnica , recorrido del proyecto , ciclo de vida y diagramas generales
Mantenimiento	Se genera desde docs/ de este repositorio con las herramientas de .github/docs/tools ; se actualiza con cada cambio del componente

CONTENIDO

Contenido

1. Propósito
2. Arquitectura del componente
3. Colecciones
4. Migración
5. Datos demo
6. Configuración
7. Pruebas
8. Operación



PARTE I

El componente

SECCIÓN 01

Propósito

En palabras simples

La base guarda todo lo que SmartPot necesita recordar: las cuentas, los cultivos reales y virtuales con su forma, sus lecturas, actuadores y órdenes, las notificaciones, los vínculos de Telegram y la configuración de las simulaciones. Rechaza los documentos que no traen lo que la API necesita, y la API se conecta con un usuario que solo puede leer y escribir su propia base.

SECCIÓN 02

Arquitectura del componente

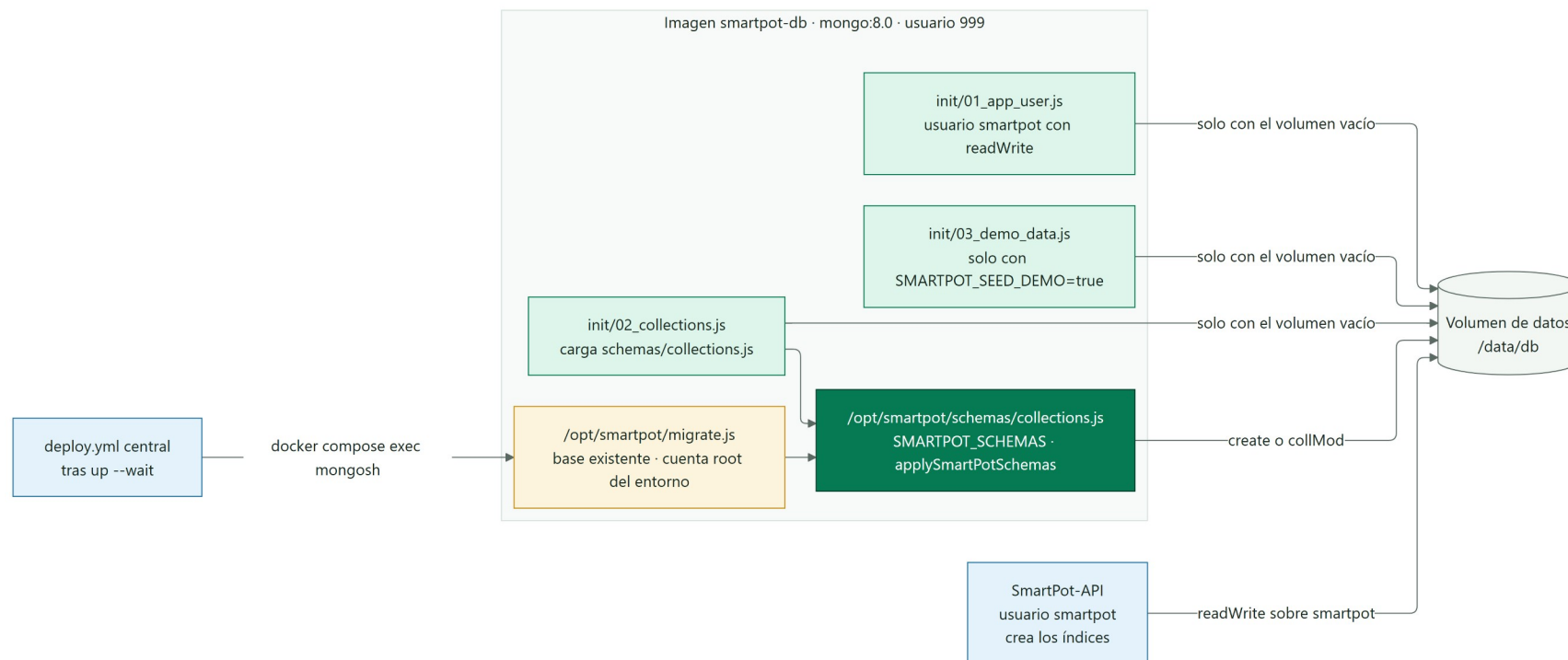


Figura 1 · SmartPot-DB por dentro



MERMAID

```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}, "layout": "elk", "elk": {"nodePlacementStrategy": "BRANDES_KOEPF", "mergeEdges":
false, "cycleBreakingStrategy": "GREEDY"}}}%
flowchart LR
    subgraph imagen["Imagen smartpot-db · mongo:8.0 · usuario 999"]
        direction TB
        init1["init/01_app_user.js<br/>usuario smartpot con readWrite"]
        init2["init/02_collections.js<br/>carga schemas/collections.js"]
        init3["init/03_demo_data.js<br/>solo con SMARTPOT_SEED_DEMO=true"]
        schemas["/opt/smartpot/schemas/collections.js<br/>SMARTPOT_SCHEMAS ·
applySmartPotSchemas"]
        migrate["/opt/smartpot/migrate.js<br/>base existente · cuenta root del entorno"]
    end
    volume[("Volumen de datos<br/>/data/db")]
    api["SmartPot-API<br/>usuario smartpot<br/>crea los índices"]
    deploy["deploy.yml central<br/>tras up --wait"]
    init1 & init2 & init3 -->|"solo con el volumen vacío"| volume
    init2 --> schemas
    migrate --> schemas
    schemas -->|"create o collMod"| volume
    deploy -->|"docker compose exec mongosh"| migrate
    api -->|"readWrite sobre smartpot"| volume
    classDef leaf fill:#DDF5EA,stroke:#067A52,color:#17261F
    classDef water fill:#E3F2FB,stroke:#1F6FA0,color:#17261F
    classDef sun fill:#FDF4DD,stroke:#C98D12,color:#17261F
    classDef clay fill:#FBE9E1,stroke:#B85A38,color:#17261F
    classDef core fill:#067A52,stroke:#0B3D2B,color:#FFFFFF
    classDef deep fill:#0B3D2B,stroke:#06281C,color:#FFFFFF
    classDef muted fill:#F2F7F4,stroke:#5B6B63,color:#17261F
    class init1,init2,init3 leaf
    class schemas core
    class migrate sun
    class volume muted
    class api,deploy water
```

SECCIÓN 03

Colecciones

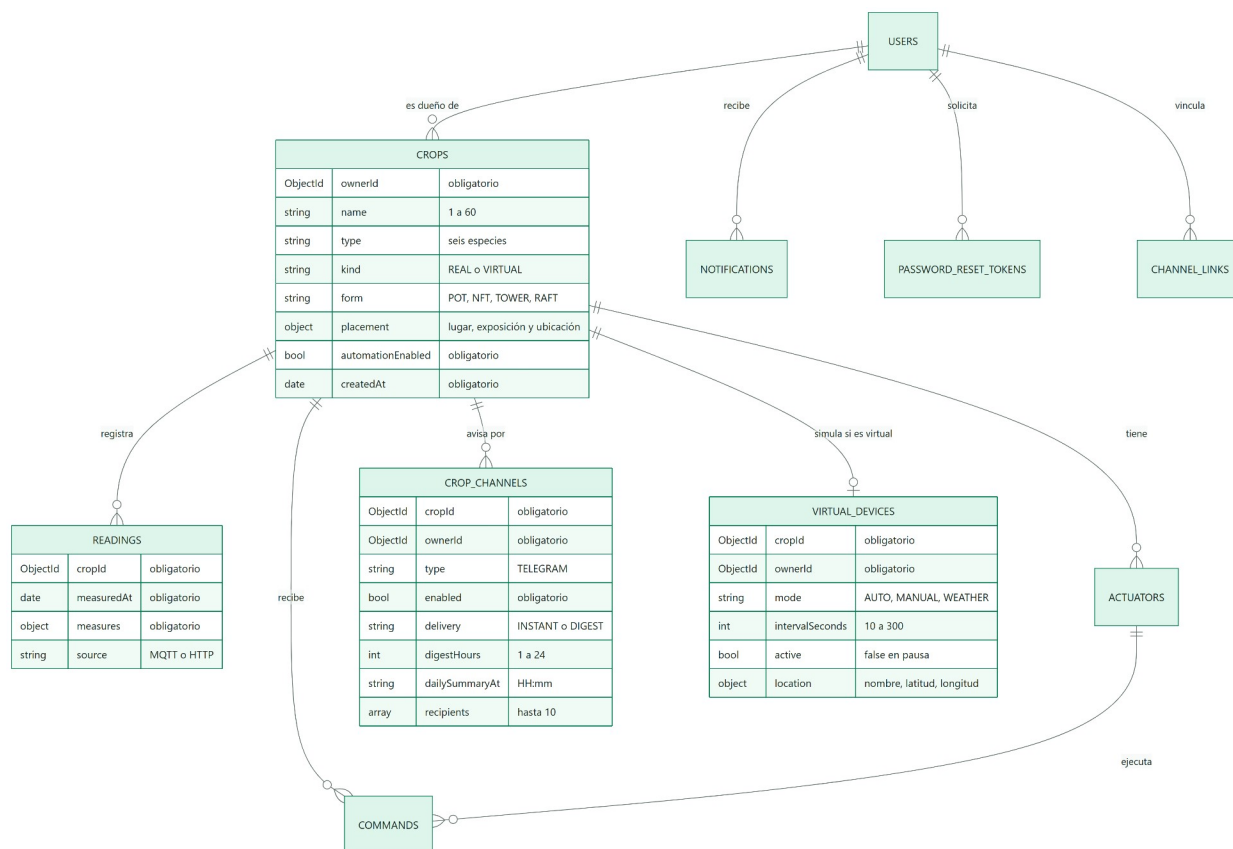


Figura 2 · Colecciones y reglas principales

MERMAID



```
%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":  
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",  
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",  
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":  
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":  
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",  
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "FFFFFF",  
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "FFFFFF",  
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",  
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "FFFFFF",  
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",  
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":  
"#067A52", "attributeBackgroundColorOdd": "FFFFFF", "attributeBackgroundColorEven":  
"#F2F7F4"}}}%
```

erDiagram

```
USERS ||--o{ CROPS : "es dueño de"  
USERS ||--o{ NOTIFICATIONS : "recibe"  
USERS ||--o{ PASSWORD_RESET_TOKENS : "solicita"  
USERS ||--o{ CHANNEL_LINKS : "vincula"  
CROPS ||--o{ READINGS : "registra"  
CROPS ||--o{ ACTUATORS : "tiene"  
CROPS ||--o{ COMMANDS : "recibe"  
CROPS ||--o{ CROP_CHANNELS : "avisa por"  
CROPS ||--o{ VIRTUAL_DEVICES : "simula si es virtual"  
ACTUATORS ||--o{ COMMANDS : "ejecuta"  
CROPS {  
    ObjectId ownerId "obligatorio"  
    string name "1 a 60"  
    string type "seis especies"  
    string kind "REAL o VIRTUAL"  
    string form "POT, NFT, TOWER, RAFT"  
    object placement "lugar, exposición y ubicación"  
    bool automationEnabled "obligatorio"  
    date createdAt "obligatorio"  
}  
READINGS {  
    ObjectId cropId "obligatorio"  
    date measuredAt "obligatorio"  
    object measures "obligatorio"  
    string source "MQTT o HTTP"  
}  
CROP_CHANNELS {  
    ObjectId cropId "obligatorio"  
    ObjectId ownerId "obligatorio"  
    string type "TELEGRAM"  
    bool enabled "obligatorio"  
    string delivery "INSTANT o DIGEST"  
    int digestHours "1 a 24"  
    string dailySummaryAt "HH:mm"  
    array recipients "hasta 10"  
}  
VIRTUAL_DEVICES {  
    ObjectId cropId "obligatorio"  
    ObjectId ownerId "obligatorio"  
    string mode "AUTO, MANUAL, WEATHER"  
    int intervalSeconds "10 a 300"
```



```
bool active "false en pausa"
object location "nombre, latitud, longitud"
}
```

COLECCIÓN	CAMPOS OBLIGATORIOS	REGLAS
users	email, passwordHash, role, createdAt	Hash BCrypt; rol USER o ADMIN
crops	ownerId, name, type, automationEnabled, createdAt	Seis especies; kind REAL o VIRTUAL; form POT, NFT, TOWER o RAFT; placement con lugar, exposición y coordenadas válidas
readings	cropId, measuredAt, measures	Origen MQTT o HTTP; TTL de un año que crea la API
actuators	cropId, type, active	Seis tipos
commands	cropId, actuatorId, actuatorType, action, status, source, createdAt	Estados y orígenes del catálogo; TTL de 180 días
notifications	userId, type, title, message, read, createdAt	TTL de 90 días
password_reset_tokens	tokenHash, userId, expiresAt	Solo el SHA-256 del token
channel_links	userId, type, address, enabled, events, linkedAt, failures	Canal TELEGRAM
crop_channels	cropId, ownerId, type, enabled, delivery	INSTANT o DIGEST; resumen cada 1 a 24 h; HH:mm; hasta 10 chats
virtual_devices	cropId, ownerId, mode, intervalSeconds, createdAt, updatedAt	Solo cultivos virtuales; active en false durante la pausa



PARTE II

Operación

SECCIÓN 04

Migración

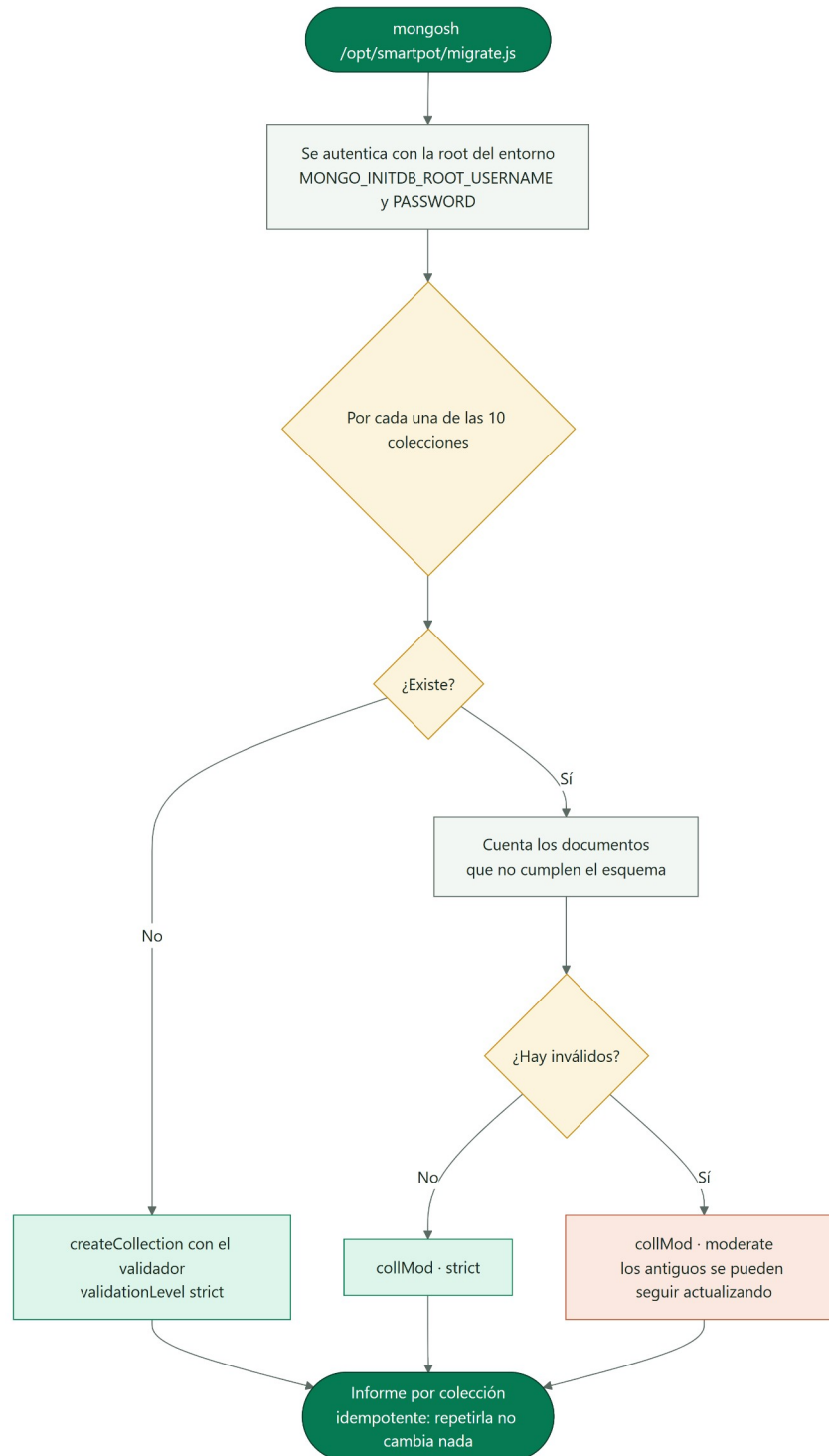


Figura 3 · Migración de una base existente



MERMAID

```

%%{init: {"theme": "base", "fontFamily": "Segoe UI, Arial, sans-serif", "themeVariables":
{"fontFamily": "Segoe UI, Arial, sans-serif", "fontSize": "15px", "primaryColor": "#DDF5EA",
"primaryTextColor": "#17261F", "primaryBorderColor": "#067A52", "secondaryColor": "#E3F2FB",
"secondaryTextColor": "#17261F", "secondaryBorderColor": "#1F6FA0", "tertiaryColor":
"#F2F7F4", "tertiaryTextColor": "#17261F", "tertiaryBorderColor": "#D5E3DC", "lineColor":
"#5B6B63", "textColor": "#17261F", "mainBkg": "#DDF5EA", "nodeBorder": "#067A52",
"clusterBkg": "#F7FAF8", "clusterBorder": "#D5E3DC", "edgeLabelBackground": "#FFFFFF",
"actorBkg": "#067A52", "actorBorder": "#0B3D2B", "actorTextColor": "#FFFFFF",
"actorLineColor": "#5B6B63", "signalColor": "#17261F", "signalTextColor": "#17261F",
"labelBoxBkgColor": "#0B3D2B", "labelBoxBorderColor": "#0B3D2B", "labelTextColor": "#FFFFFF",
"loopTextColor": "#0B3D2B", "noteBkgColor": "#FDF4DD", "noteBorderColor": "#C98D12",
"noteTextColor": "#17261F", "activationBkgColor": "#DDF5EA", "activationBorderColor":
"#067A52", "attributeBackgroundColorOdd": "#FFFFFF", "attributeBackgroundColorEven":
"#F2F7F4"}}}%
flowchart TB
    start(["mongosh /opt/smartpot/migrate.js"]) --> auth["Se autentica con la root del
entorno<br/>MONGO_INITDB_ROOT_USERNAME y PASSWORD"]
    auth --> each["Por cada una de las 10 colecciones"]
    each --> exists["¿Existe?"]
    exists -->|No| create["createCollection con el validador<br/>validationLevel strict"]
    exists -->|Sí| count["Cuenta los documentos<br/>que no cumplen el esquema"]
    count --> invalid["¿Hay inválidos?"]
    invalid -->|No| strict["collMod · strict"]
    invalid -->|Sí| moderate["collMod · moderate<br/>los antiguos se pueden seguir
actualizando"]
    create & strict & moderate --> report(["Informe por colección<br/>idempotente: repetirla no
cambia nada"])
    classDef leaf fill:#DDF5EA,stroke:#067A52,color:#17261F
    classDef water fill:#E3F2FB,stroke:#1F6FA0,color:#17261F
    classDef sun fill:#FDF4DD,stroke:#C98D12,color:#17261F
    classDef clay fill:#FBE9E1,stroke:#B85A38,color:#17261F
    classDef core fill:#067A52,stroke:#0B3D2B,color:#FFFFFF
    classDef deep fill:#0B3D2B,stroke:#06281C,color:#FFFFFF
    classDef muted fill:#F2F7F4,stroke:#5B6B63,color:#17261F
    class start,report core
    class each,exists,invalid sun
    class auth,count muted
    class create,strict leaf
    class moderate clay

```

Los scripts de `init/` solo corren con el volumen vacío. Una base existente recibe los esquemas nuevos con la migración, que el despliegue central ejecuta después de `up --wait`; a mano: `docker exec smartpot-db mongosh --quiet /opt/smartpot/migrate.js`.



SECCIÓN 05

Datos demo

Con `SMARTPOT_SEED_DEMO=true` se crea la cuenta `demo@smartpot.app` (contraseña pública de la demo) con dos cultivos ****reales**** que publica el simulador: lechugas en tubos NFT con modo automático y tomates cherry en maceta, cada uno con 48 horas de lecturas. Nunca se activa en producción.

SECCIÓN 06

Configuración

VARIABLE	USO
MONGO_INITDB_ROOT_USERNAME, MONGO_INITDB_ROOT_PASSWORD	Cuenta administradora, solo para inicializar y migrar
SMARTPOT_DB_NAME, SMARTPOT_DB_USERNAME, SMARTPOT_DB_PASSWORD	Base y usuario de la aplicación
SMARTPOT_SEED_DEMO	Datos demo (false por defecto)

SECCIÓN 07

Pruebas

`sh tests/validate.sh smartpot-db:ci true` levanta la imagen y comprueba las diez colecciones con sus validadores (el lugar del cultivo y los avisos por cultivo incluidos), los permisos del usuario de la aplicación, los datos demo y la migración sobre una base anterior (una colección sin validador y otra con un documento antiguo), incluida su idempotencia. En Git Bash de Windows: `MSYS_NO_PATHCONV=1`.

SECCIÓN 08

Operación

TAREA	CÓMO
Imagen	<code>ghcr.io/smartpotttech/smartpot-db: usuario 999, solo lectura con tmpfs en /tmp</code>
Respaldo	<code>mongodump</code> comprimido desde el servidor; restauración con <code>mongorestore --drop --archive --gzip</code>
Red	Solo la red interna; nunca se publica el puerto 27017
Despliegue	Cada cambio en <code>main</code> pasa por el CI, publica la imagen y pide el despliegue central de <code>.github</code> , que migra tras actualizar